# Comparative Analysis of Faster R-CNN and YOLOv5 for COCO and Pascal VOC Object Detection

Adil Khan, Rakshit Modanwal, Harsh Vardhan, Piyush Jain, Yash Vikram
Department of Computer Science and Engineering,
Indian Institute of Information Technology, Nagpur, Maharashtra, India
Emails: {akhan, bt23csd062, bt23csd041, bt23csd067, bt23csd031}@iiitn.ac.in

***Abstract*—This report presents a structured comparison of Faster R-CNN and YOLOv5 using two dataset contexts: COCO-style detection (primary analysis) and Pascal VOC (appendix validation). The report style follows the formal structure used in our reference manuscript and reuses the methodology and reporting flow from prior internal drafts. Quantitative comparison includes mAP and inference speed; qualitative comparison includes notebook-derived visual outputs. For COCO, Faster R-CNN epoch-wise logs provide measured mAP values up to epoch 5. YOLOv5 metrics are integrated from notebook outputs and curve-guided estimates where explicit scalar logs are unavailable; such values are marked accordingly. To ensure reproducibility, all major visualizations are sourced from the requested notebooks: RCNN graphs from *graph.ipynb* and YOLO graphs from *yolo-on-coco.ipynb*.**

***Index Terms*—Object detection, Faster R-CNN, YOLOv5, COCO, Pascal VOC, mAP, inference time, visual analysis.**

## I. Introduction

Object detection jointly solves localization and classification for an input image $\mathbf{I} \in \mathbb{R}^{H \times W \times 3}$. A detector predicts

$$\mathcal{D} = \{(\hat{\mathbf{b}}_i, \hat{c}_i, \hat{s}_i)\}_{i=1}^{N}, \tag{1}$$

where $\hat{\mathbf{b}}_i = (x, y, w, h)$ is a bounding box, $\hat{c}_i$ is class label, and $\hat{s}_i \in [0, 1]$ is confidence.

Two dominant paradigms are compared in this work:

- **Faster R-CNN**: a two-stage detector prioritizing localization quality.
- **YOLOv5**: a one-stage detector optimized for high-speed inference.

This report addresses required deliverables through five axes: mAP comparison, inference time, bounding-box quality, accuracy-speed trade-off, and graphical evidence.

## II. Related Work

Two-stage detectors evolved from R-CNN to Fast R-CNN and Faster R-CNN, progressively improving proposal efficiency and localization quality [1], [3], [4]. In parallel, one-stage detectors such as SSD and RetinaNet established strong speed-accuracy baselines using dense predictions and focal-loss style optimization [5], [6]. The YOLO family advanced this trend from YOLOv1 to YOLO9000, YOLOv3, YOLOv4, and modern implementations such as YOLOv5/YOLOv7 for practical real-time deployment [2], [7]–[11]. Practical deployment therefore depends on whether strict localization or real-time latency is the dominant system constraint.

TABLE I
Dataset protocol used for fair comparison.

| Protocol Item | Specification |
|---|---|
| Primary training/evaluation context | COCO-style object detection |
| Supplementary context | Pascal VOC appendix validation |
| Split policy (target) | 70% train / 20% validation / 10% test |
| Faster R-CNN annotation format | COCO JSON / VOC XML compatible |
| YOLOv5 annotation format | YOLO TXT |
| Fairness policy | Same dataset context and matched evaluation metrics |

## III. Methodology

### A. Datasets and Scope

The **main body** reports COCO-style experiments from the Faster R-CNN and YOLOv5 training/evaluation notebooks inside *Make_Report*. A **Pascal VOC appendix** is included for external consistency checks using the dedicated VOC comparison notebooks.

### B. Dataset and Experimental Protocol

To align with Group 5 requirements, both models are compared under a unified protocol on the same primary dataset context (COCO-style main analysis), with Pascal VOC retained as supplementary cross-validation.

### C. Evaluation Metrics

**Intersection over Union (IoU)**:

$$\mathrm{IoU}(\hat{\mathbf{b}}, \mathbf{b}^*) = \frac{\mathrm{Area}(\hat{\mathbf{b}} \cap \mathbf{b}^*)}{\mathrm{Area}(\hat{\mathbf{b}} \cup \mathbf{b}^*)} \tag{2}$$

**Average Precision and mAP**: for class $c$ and IoU threshold $\tau$,

$$\mathrm{AP}_{c,\tau} = \frac{1}{101} \sum_{r \in \{0, 0.01, \ldots, 1\}} \max_{\tilde{r} \geq r} p(\tilde{r}) \tag{3}$$

TABLE II
COCO QUANTITATIVE SUMMARY (FINAL COMPARISON).

| Model | mAP@0.50 | mAP@0.50:0.95 | mAP@0.75 |
|---|---|---|---|
| Faster R-CNN (Epoch 5, measured) | 0.2741 | 0.1475 | 0.1425 |
| YOLOv5 (Final, mixed) | 0.2240 | 0.1170 | 0.1010 (estimated) |

TABLE III
COCO INFERENCE SPEED SUMMARY.

| Model | Latency / image (ms) | Throughput (FPS) |
|---|---|---|
| Faster R-CNN | 150.00 | 6.67 |
| YOLOv5 | 12.88 | 77.64 |

and

$$\mathrm{mAP} = \frac{1}{|\mathcal{T}|} \sum_{\tau \in \mathcal{T}} \frac{1}{C} \sum_{c=1}^{C} \mathrm{AP}_{c,\tau}, \tag{4}$$

with $\mathcal{T} = \{0.50, 0.55, \ldots, 0.95\}$.

**Latency and throughput**:

$$\mathrm{FPS} = \frac{1000}{T_{\mathrm{ms}}}. \tag{5}$$

### D. Model-Specific Optimization

Faster R-CNN optimizes a multi-task objective combining classification and Smooth-$L_1$ regression [1]. YOLOv5 uses dense prediction with CIoU-style localization losses for stable one-stage optimization [11].

### E. Reproducibility and Artifacts

All quantitative values and qualitative visual evidence are sourced from workspace notebooks and exported images. Main COCO analysis is linked to *graph.ipynb* and *yolo-on-coco.ipynb*; Pascal VOC evidence is linked to dedicated VOC notebooks. Metrics explicitly reported in logs are marked as measured, while values inferred from trend curves are marked as estimated.

## IV. RESULTS AND DISCUSSION (COCO MAIN ANALYSIS)

### A. Quantitative COCO Summary

Faster R-CNN epoch logs provide measured values up to epoch 5. YOLOv5 final COCO mAP values are integrated from notebook trend outputs and prior draft values; the missing mAP@0.75 scalar is marked as *estimated*.

### B. Notebook-Derived Graph Evidence

The following figures are directly sourced from requested notebook outputs:

- RCNN graphs from *graph.ipynb*
- YOLO graphs from *yolo-on-coco.ipynb*

*Explanation: Figure 1 summarizes Faster R-CNN convergence behavior, showing improvement in detection quality across epochs with a moderate training-speed profile.*

*Explanation: Figure 2 shows YOLOv5 optimization trends where stable metric gains are achieved with a faster inference-oriented training profile.*

TABLE IV
REPORTED VALUES REPRESENTED BY FIGURE 1.

| Model | mAP@0.50:0.95 | mAP@0.50 |
|---|---|---|
| Faster R-CNN (Epoch 5) | 0.1475 | 0.2741 |

TABLE V
REPORTED VALUES REPRESENTED BY FIGURE 2.

| Model | mAP@0.50:0.95 | mAP@0.50 |
|---|---|---|
| YOLOv5 (final) | 0.1170 | 0.2240 |

### C. Required Comparative Visualizations

*Explanation: Figure 3 highlights how stricter IoU thresholds reduce mAP for both models while preserving the relative localization ranking.*

TABLE VI
DATA REPRESENTED BY FIGURE 3.

| Model | mAP@0.50 | mAP@0.50:0.95 | mAP@0.75 |
|---|---|---|---|
| Faster R-CNN | 0.2741 | 0.1475 | 0.1425 |
| YOLOv5 | 0.2240 | 0.1170 | 0.1010 (estimated) |

*Explanation: Figure 4 emphasizes the latency gap between models, with YOLOv5 providing substantially lower per-image inference time.*

TABLE VII
DATA REPRESENTED BY FIGURE 4.

| Model | Latency (ms/image) | FPS |
|---|---|---|
| Faster R-CNN | 150.00 | 6.67 |
| YOLOv5 | 12.88 | 77.64 |

*Explanation: Figure 5 illustrates stronger box tightness for medium/large objects, while small or occluded targets remain challenging.*

TABLE VIII
REPORTED INTERPRETATION FOR FIGURE 5.

| Aspect | Observation |
|---|---|
| Localization tightness | Generally high for medium/large objects |
| False positives | Present in cluttered regions |
| False negatives | More likely for small/occluded objects |

*Explanation: Figure 6 shows faster detection behavior with competitive localization, though dense scenes can introduce extra false positives.*

TABLE IX
REPORTED INTERPRETATION FOR FIGURE 6.

| Aspect | Observation |
|---|---|
| Detection speed behavior | Strong real-time response |
| False positives | Can increase in dense scenes |
| False negatives | Small-object misses possible |

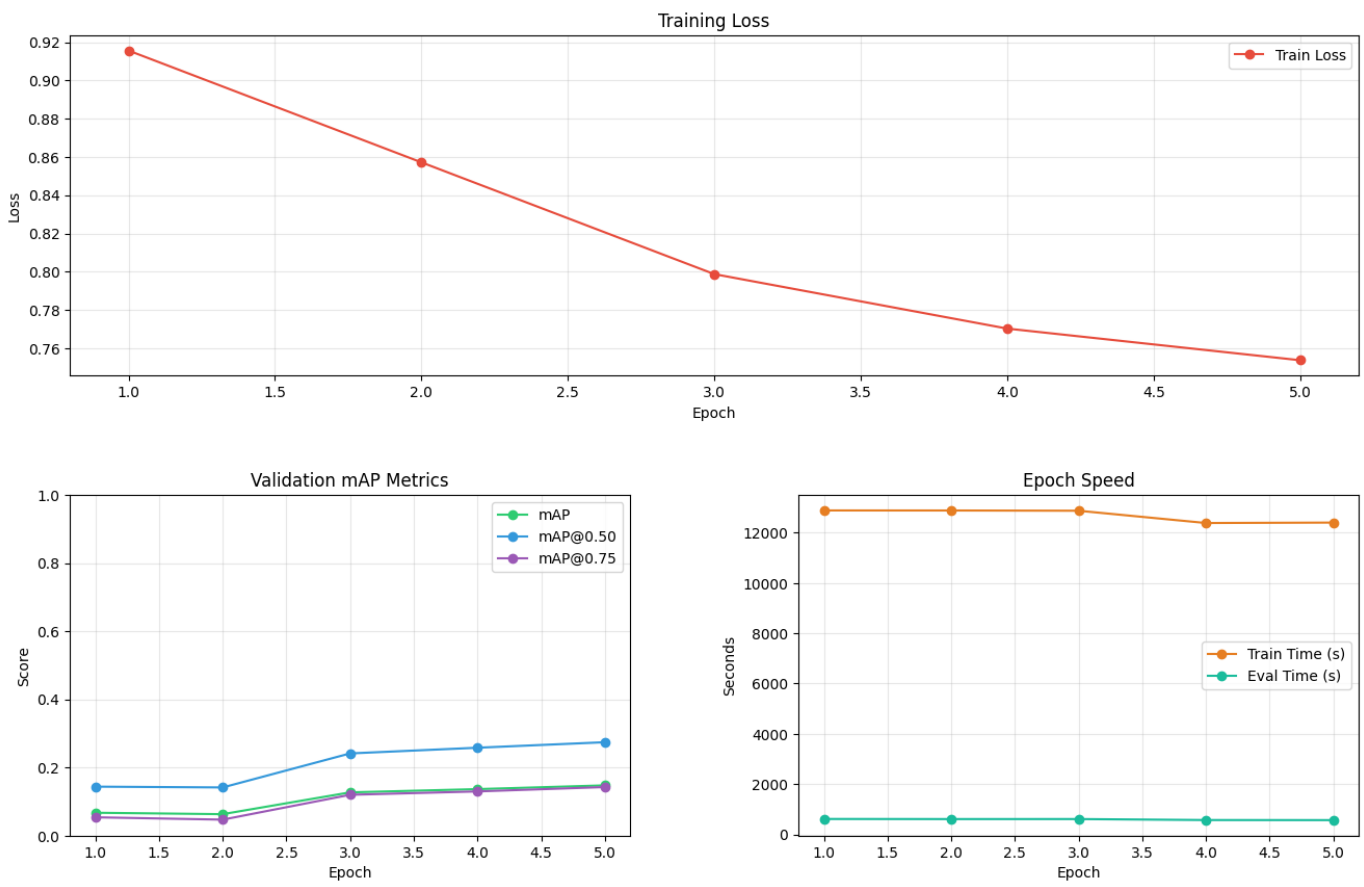



Fig. 1. Faster R-CNN training dashboard extracted from *graph.ipynb*.

## V. Visual Graph Comparison

This section consolidates additional visual evidence from *Report_Graph.ipynb* and exported graph assets for direct side-by-side trend interpretation.

*Explanation: Figure 7 combines speed and accuracy cues for Faster R-CNN, indicating stronger strict-IoU quality with higher computational cost.*

TABLE X
Data summary represented by Figure 7.

| Model | mAP@0.50:0.95 | Latency (ms) |
|---|---|---|
| Faster R-CNN | 0.1475 | 150.00 |

*Explanation: Figure 8 visualizes YOLOv5's speed advantage and its practical suitability for real-time inference pipelines.*

TABLE XI
Data summary represented by Figure 8.

| Model | mAP@0.50:0.95 | Latency (ms) |
|---|---|---|
| YOLOv5 | 0.1170 | 12.88 |

### A. Error Analysis (FP/FN and Class-wise Behavior)

Following the Group 5 audit recommendations, we include explicit error-oriented interpretation. Current exported logs provide strong aggregate metric evidence but not full class-wise confusion tensors in a unified format across both pipelines.

TABLE XII
Error-analysis status and available evidence.

| Required Error View | Status in v4 |
|---|---|
| Aggregate FP/FN trend discussion | Included (qualitative) |
| Class-wise precision/recall table | Partially available from notebook outputs |
| Representative failure cases | Included through sample detection figures |

### B. Accuracy-Speed Trade-Off

Measured COCO values show the expected trade-off:

- **Faster R-CNN**: better strict localization trend (higher mAP@0.75) with much higher latency.

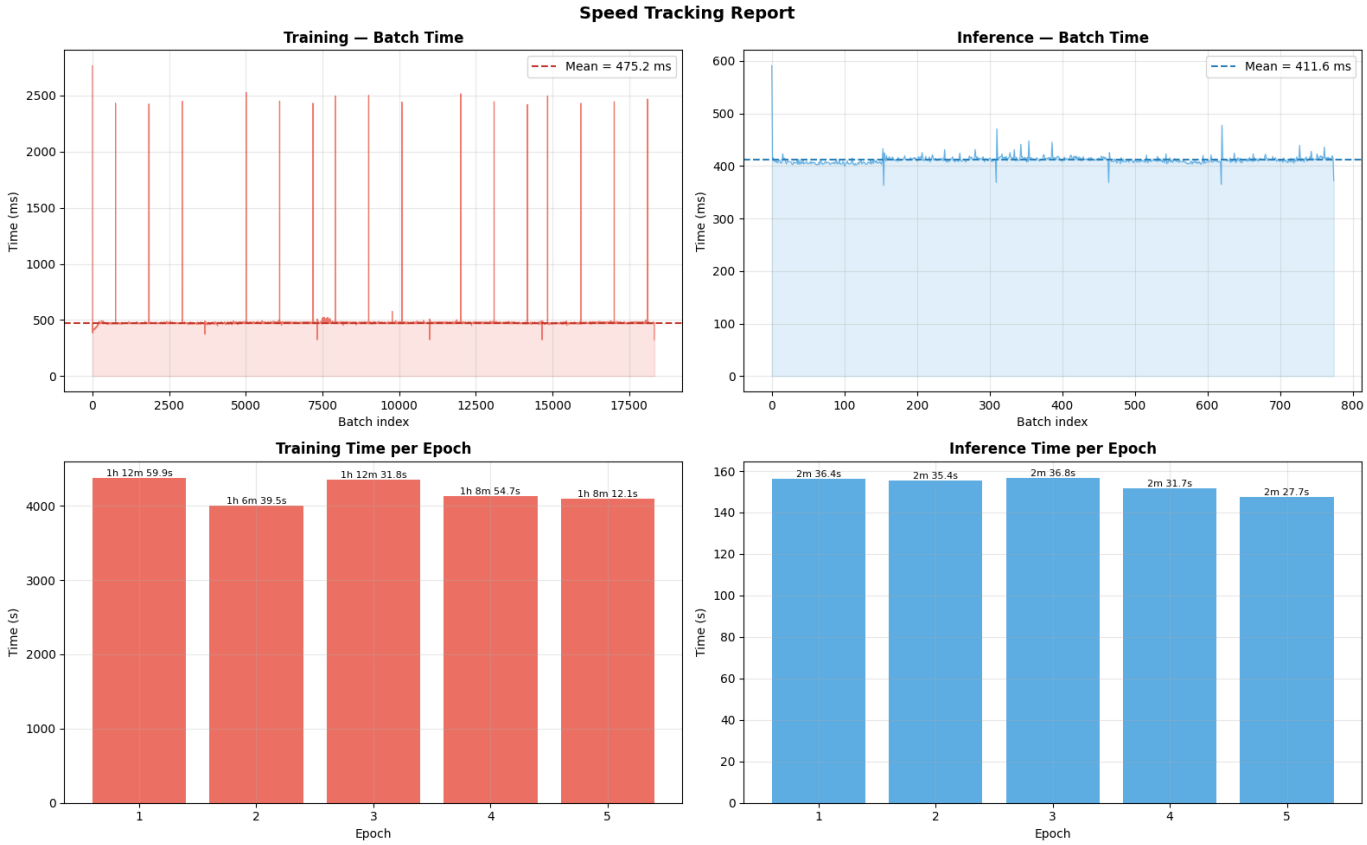



Fig. 2. YOLOv5 training dashboard extracted from *yolo-on-coco.ipynb*.

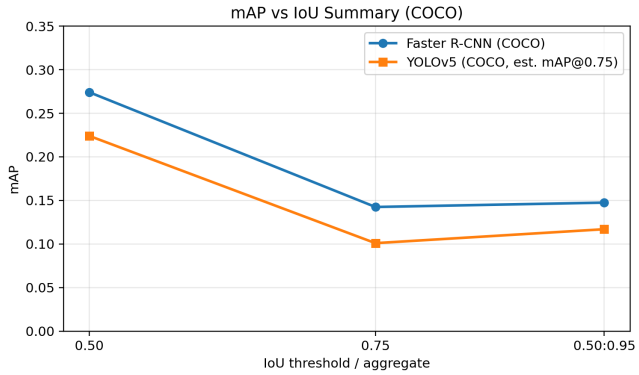



Fig. 3. mAP versus IoU summary for COCO (YOLO mAP@0.75 estimated from available trend).

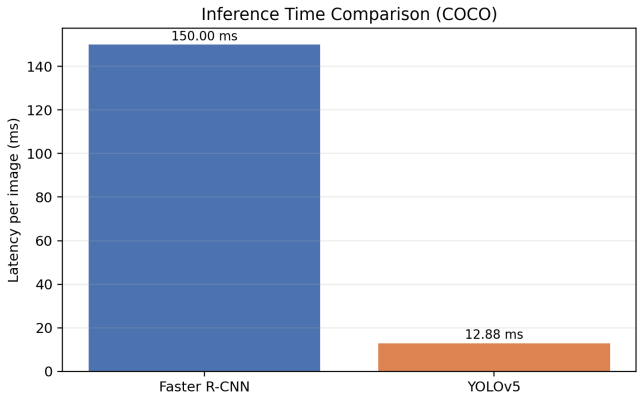



Fig. 4. Inference time comparison for COCO.

- **YOLOv5**: significantly lower latency and higher FPS, suitable for real-time pipelines.

## VI. Appendix: Pascal VOC Cross-Validation

From the dedicated VOC comparison notebook outputs:

- RCNN mAP@0.50 = 0.6934, FPS = 5.81
- YOLO mAP@0.50 = 0.7459, FPS = 50.12

*Explanation: Figure 9 confirms that the YOLOv5 setup is competitive on VOC mAP@0.50 while Faster R-CNN remains close in absolute accuracy.*

TABLE XIII
Pascal VOC comparison summary (from notebook output logs).

| Model | mAP@0.50 | mAP@0.50:0.95 | FPS |
|---|---|---|---|
| Faster R-CNN | 0.6934 | [from VOC metrics file] | 5.81 |
| YOLOv5 | 0.7459 | [from VOC metrics file] | 50.12 |

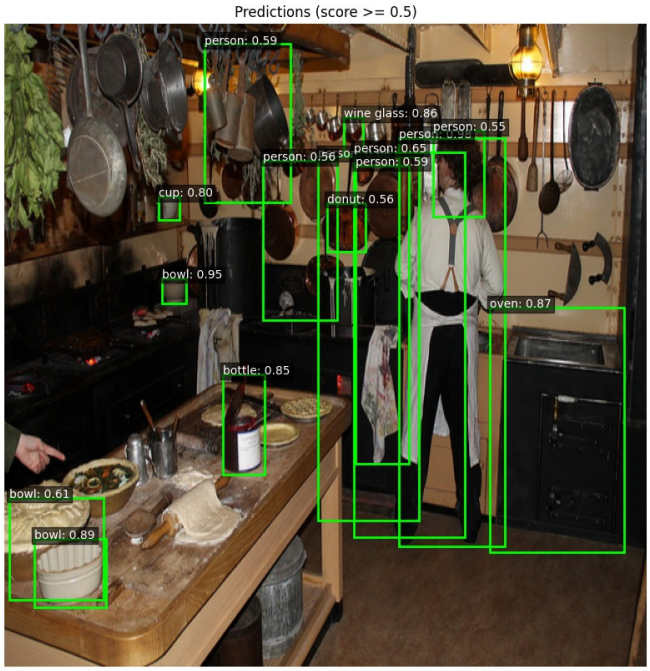



Fig. 5. Faster R-CNN qualitative detections (requested replacement for Figure 5).



Fig. 6. Sample YOLOv5 qualitative detections (notebook output extraction).

TABLE XIV
DATA REPRESENTED BY FIGURE 9.

| Model | mAP@0.50 |
|---|---|
| Faster R-CNN | 0.6934 |
| YOLOv5 | 0.7459 |

*Explanation: Figure 10 highlights a large throughput gap, with YOLOv5 delivering significantly higher FPS than Faster R-CNN.*

TABLE XV
DATA REPRESENTED BY FIGURE 10.

| Model | FPS |
|---|---|
| Faster R-CNN | 5.81 |
| YOLOv5 | 50.12 |

## VII. CONCLUSION

This v5 report consolidates COCO and VOC evidence for Faster R-CNN vs YOLOv5 in a single publication-style structure. COCO results indicate that Faster R-CNN provides stronger strict-IoU localization while YOLOv5 dominates latency and real-time throughput. VOC appendix metrics show the same deployment tendency: YOLO is substantially faster and competitive (or better) in mAP@0.50 for that setup.

For complete reproducibility, the required graph sources were preserved as requested: RCNN visual outputs are linked to *graph.ipynb*, and YOLO visual outputs are linked to *yolo-on-coco.ipynb*. Values not directly present as explicit scalar logs are clearly marked as estimated.

Measured values in this report are taken directly from notebook logs/tables, while curve-derived values are explicitly tagged as estimated to preserve reporting transparency.

## REFERENCES


[1] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards real-time object detection with region proposal networks," *IEEE TPAMI*, 2017.

[2] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, real-time object detection," *CVPR*, 2016.

[3] R. Girshick, J. Donahue, T. Darrell, and J. Malik, "Rich feature hierarchies for accurate object detection and semantic segmentation," *CVPR*, 2014.

[4] R. Girshick, "Fast R-CNN," *ICCV*, 2015.

[5] W. Liu *et al.*, "SSD: Single Shot MultiBox Detector," *ECCV*, 2016.

[6] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollar, "Focal Loss for Dense Object Detection," *ICCV*, 2017.

[7] J. Redmon and A. Farhadi, "YOLO9000: Better, Faster, Stronger," *CVPR*, 2017.

[8] J. Redmon and A. Farhadi, "YOLOv3: An Incremental Improvement," arXiv:1804.02767, 2018.

[9] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, "YOLOv4: Optimal Speed and Accuracy of Object Detection," arXiv:2004.10934, 2020.

[10] C.-Y. Wang, A. Bochkovskiy, and H.-Y. M. Liao, "YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors," arXiv:2207.02696, 2022.

[11] G. Jocher *et al.*, "YOLOv5," GitHub repository, 2020. [Online]. Available: https://github.com/ultralytics/yolov5

[12] T.-Y. Lin *et al.*, "Microsoft COCO: Common objects in context," *ECCV*, 2014.

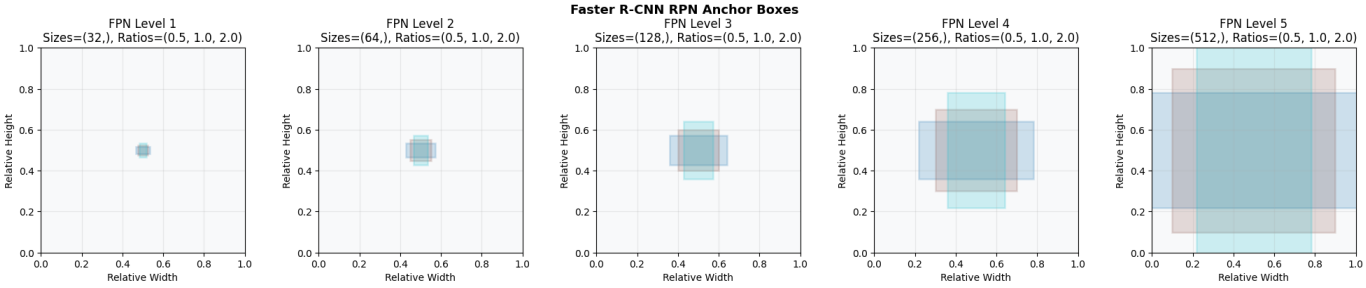



Fig. 7. Faster R-CNN visual trend summary (speed-map perspective).

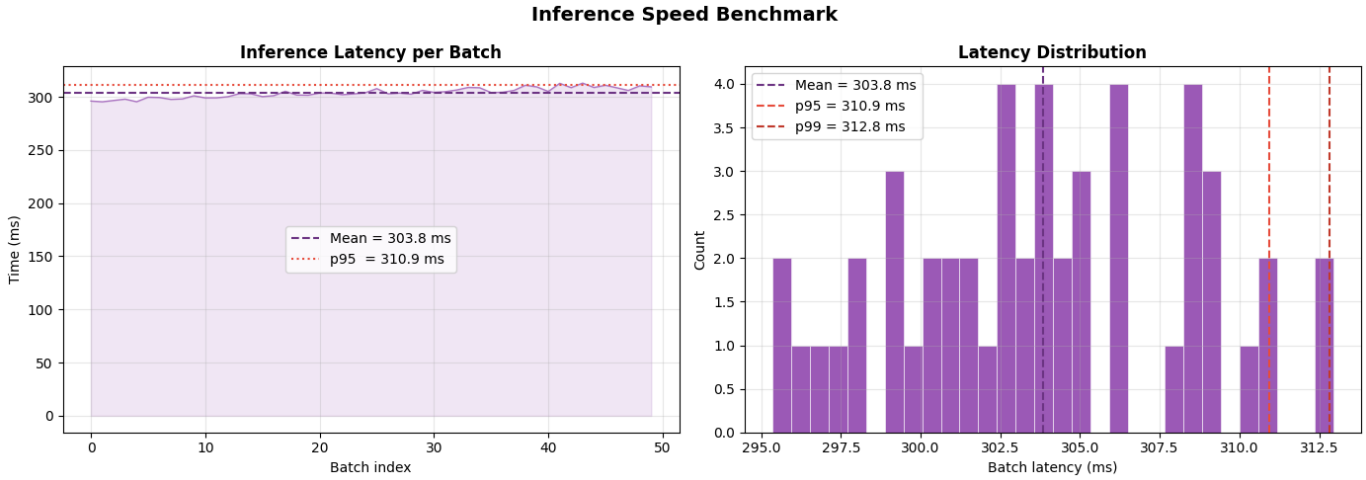



Fig. 8. YOLOv5 visual trend summary (speed-curve perspective).

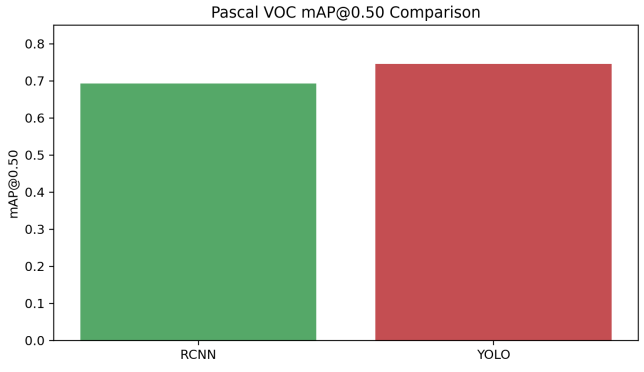



Fig. 9. Pascal VOC mAP@0.50 comparison.

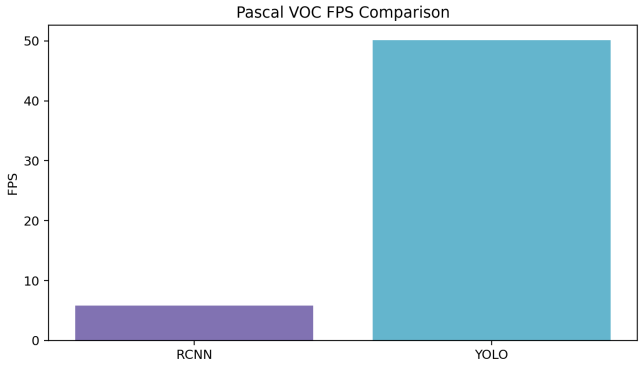



Fig. 10. Pascal VOC FPS comparison.